



Overview

Ask a home buyer to describe their dream house, and they probably won't begin with the height of the basement ceiling or the proximity to an east-west railroad. But this playground competition's dataset proves that much more influences price negotiations than the number of bedrooms or a white-picket fence.

With 79 explanatory variables describing (almost) every aspect of residential homes in Ames, Iowa, this competition challenges you to predict the final price of each home

Acknowledgments

The Ames Housing dataset was compiled by Dean De Cock for use in data science education. It's an incredible alternative for data scientists looking for a modernized and expanded version of the often cited Boston Housing dataset.

You can find the dataset [here](https://www.kaggle.com/c/house-prices-advanced-regression-techniques/data) (<https://www.kaggle.com/c/house-prices-advanced-regression-techniques/data>).

Files

1. train.csv - the training set
2. test.csv - the test set
3. data_description.txt - full description of each column, originally prepared by Dean De Cock but lightly edited to match the column names used here

So lets begin with complete EDA...

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import pickle
```

Load Data

```
train = pd.read_csv('train.csv')
test = pd.read_csv('test.csv')
```

```
print("Train Shape: ",train.shape)
print("Test Shape: ",test.shape)
```

Train Shape: (1460, 81)

Test Shape: (1459, 80)

train has 81 columns (79 features + id and target SalePrice) and 1460 entries
test has 80 columns (79 features + id) and 1459 entries

```
print(train.info())
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1460 entries, 0 to 1459
Data columns (total 81 columns):
Id                1460 non-null int64
MSSubClass        1460 non-null int64
MSZoning          1460 non-null object
LotFrontage       1201 non-null float64
LotArea           1460 non-null int64
Street            1460 non-null object
Alley             91 non-null object
LotShape          1460 non-null object
LandContour       1460 non-null object
Utilities         1460 non-null object
LotConfig         1460 non-null object
LandSlope         1460 non-null object
Neighborhood      1460 non-null object
Condition1        1460 non-null object
Condition2        1460 non-null object
BldgType          1460 non-null object
HouseStyle        1460 non-null object
OverallQual       1460 non-null int64
OverallCond       1460 non-null int64
YearBuilt         1460 non-null int64
YearRemodAdd      1460 non-null int64
RoofStyle         1460 non-null object
RoofMat1          1460 non-null object
Exterior1st       1460 non-null object
Exterior2nd       1460 non-null object
MasVnrType        1452 non-null object
MasVnrArea        1452 non-null float64
ExterQual         1460 non-null object
ExterCond         1460 non-null object
Foundation        1460 non-null object
BsmtQual          1423 non-null object
BsmtCond          1423 non-null object
BsmtExposure      1422 non-null object
BsmtFinType1      1423 non-null object
BsmtFinSF1        1460 non-null int64
BsmtFinType2      1422 non-null object
BsmtFinSF2        1460 non-null int64
BsmtUnfSF         1460 non-null int64
TotalBsmtSF       1460 non-null int64
Heating           1460 non-null object
HeatingQC         1460 non-null object
CentralAir        1460 non-null object
Electrical        1459 non-null object
1stFlrSF          1460 non-null int64

```

```
2ndFlrSF      1460 non-null int64
LowQualFinSF  1460 non-null int64
GrLivArea     1460 non-null int64
BsmtFullBath  1460 non-null int64
BsmtHalfBath  1460 non-null int64
FullBath      1460 non-null int64
HalfBath      1460 non-null int64
BedroomAbvGr  1460 non-null int64
KitchenAbvGr  1460 non-null int64
KitchenQual   1460 non-null object
TotRmsAbvGrd  1460 non-null int64
Functional    1460 non-null object
Fireplaces    1460 non-null int64
FireplaceQu   770 non-null object
GarageType     1379 non-null object
GarageYrBlt    1379 non-null float64
GarageFinish   1379 non-null object
GarageCars     1460 non-null int64
GarageArea     1460 non-null int64
GarageQual     1379 non-null object
GarageCond     1379 non-null object
PavedDrive     1460 non-null object
WoodDeckSF     1460 non-null int64
OpenPorchSF    1460 non-null int64
EnclosedPorch  1460 non-null int64
3SsnPorch      1460 non-null int64
ScreenPorch    1460 non-null int64
PoolArea       1460 non-null int64
PoolQC         7 non-null object
Fence          281 non-null object
MiscFeature    54 non-null object
MiscVal        1460 non-null int64
MoSold         1460 non-null int64
YrSold         1460 non-null int64
SaleType       1460 non-null object
SaleCondition  1460 non-null object
SalePrice      1460 non-null int64
dtypes: float64(3), int64(35), object(43)
memory usage: 924.0+ KB
None
```

```
print(test.info())
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1459 entries, 0 to 1458
Data columns (total 80 columns):
Id                1459 non-null int64
MSSubClass        1459 non-null int64
MSZoning          1455 non-null object
LotFrontage       1232 non-null float64
LotArea           1459 non-null int64
Street            1459 non-null object
Alley             107 non-null object
LotShape          1459 non-null object
LandContour       1459 non-null object
Utilities          1457 non-null object
LotConfig         1459 non-null object
LandSlope         1459 non-null object
Neighborhood      1459 non-null object
Condition1        1459 non-null object
Condition2        1459 non-null object
BldgType          1459 non-null object
HouseStyle        1459 non-null object
OverallQual       1459 non-null int64
OverallCond       1459 non-null int64
YearBuilt         1459 non-null int64
YearRemodAdd      1459 non-null int64
RoofStyle         1459 non-null object
RoofMat1          1459 non-null object
Exterior1st       1458 non-null object
Exterior2nd       1458 non-null object
MasVnrType        1443 non-null object
MasVnrArea        1444 non-null float64
ExterQual         1459 non-null object
ExterCond         1459 non-null object
Foundation        1459 non-null object
BsmtQual          1415 non-null object
BsmtCond          1414 non-null object
BsmtExposure      1415 non-null object
BsmtFinType1      1417 non-null object
BsmtFinSF1        1458 non-null float64
BsmtFinType2      1417 non-null object
BsmtFinSF2        1458 non-null float64
BsmtUnfSF         1458 non-null float64
TotalBsmtSF       1458 non-null float64
Heating           1459 non-null object
HeatingQC         1459 non-null object
CentralAir        1459 non-null object
Electrical        1459 non-null object
1stFlrSF          1459 non-null int64

```

```

2ndFlrSF          1459 non-null int64
LowQualFinSF      1459 non-null int64
GrLivArea         1459 non-null int64
BsmtFullBath      1457 non-null float64
BsmtHalfBath      1457 non-null float64
FullBath          1459 non-null int64
HalfBath          1459 non-null int64
BedroomAbvGr      1459 non-null int64
KitchenAbvGr      1459 non-null int64
KitchenQual       1458 non-null object
TotRmsAbvGrd      1459 non-null int64
Functional         1457 non-null object
Fireplaces        1459 non-null int64
FireplaceQu       729 non-null object
GarageType        1383 non-null object
GarageYrBlt       1381 non-null float64
GarageFinish      1381 non-null object
GarageCars        1458 non-null float64
GarageArea        1458 non-null float64
GarageQual        1381 non-null object
GarageCond        1381 non-null object
PavedDrive        1459 non-null object
WoodDeckSF        1459 non-null int64
OpenPorchSF       1459 non-null int64
EnclosedPorch     1459 non-null int64
3SsnPorch         1459 non-null int64
ScreenPorch       1459 non-null int64
PoolArea          1459 non-null int64
PoolQC            3 non-null object
Fence             290 non-null object
MiscFeature       51 non-null object
MiscVal           1459 non-null int64
MoSold            1459 non-null int64
YrSold            1459 non-null int64
SaleType          1458 non-null object
SaleCondition     1459 non-null object
dtypes: float64(11), int64(26), object(43)
memory usage: 912.0+ KB
None

```

Check for NULL values

Train Data

```
print(train.isnull().sum())
```

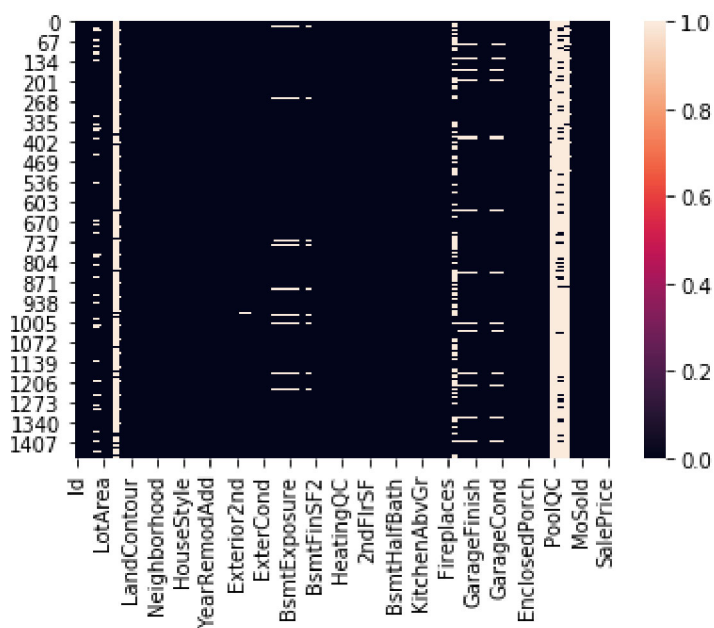
Id	0
MSSubClass	0
MSZoning	0
LotFrontage	259
LotArea	0
Street	0
Alley	1369
LotShape	0
LandContour	0
Utilities	0
LotConfig	0
LandSlope	0
Neighborhood	0
Condition1	0
Condition2	0
BldgType	0
HouseStyle	0
OverallQual	0
OverallCond	0
YearBuilt	0
YearRemodAdd	0
RoofStyle	0
RoofMatl	0
Exterior1st	0
Exterior2nd	0
MasVnrType	8
MasVnrArea	8
ExterQual	0
ExterCond	0
Foundation	0
...	
BedroomAbvGr	0
KitchenAbvGr	0
KitchenQual	0
TotRmsAbvGrd	0
Functional	0
Fireplaces	0
FireplaceQu	690
GarageType	81
GarageYrBlt	81
GarageFinish	81
GarageCars	0
GarageArea	0
GarageQual	81
GarageCond	81
PavedDrive	0
WoodDeckSF	0

```

OpenPorchSF      0
EnclosedPorch    0
3SsnPorch        0
ScreenPorch      0
PoolArea         0
PoolQC           1453
Fence            1179
MiscFeature      1406
MiscVal          0
MoSold           0
YrSold           0
SaleType         0
SaleCondition    0
SalePrice        0
Length: 81, dtype: int64

```

```
sns.heatmap(train.isnull())
```



Test Data

```
print(test.isnull().sum())
```

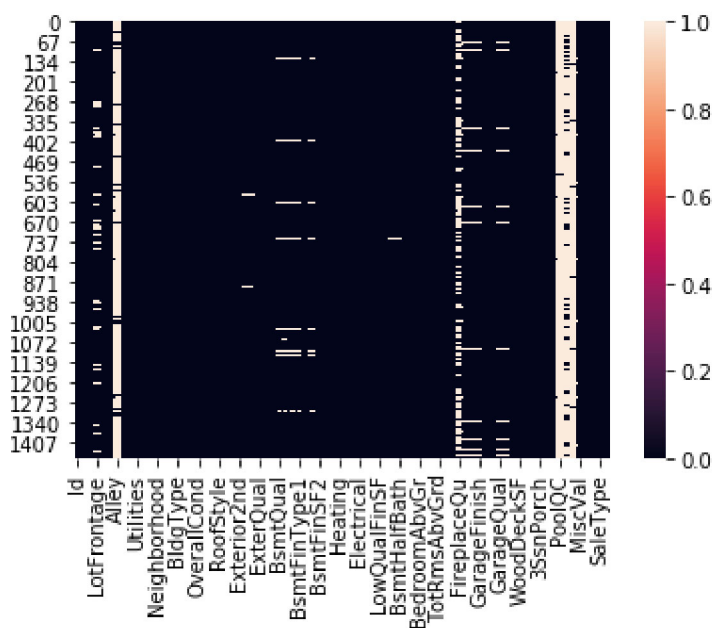
Id	0
MSSubClass	0
MSZoning	4
LotFrontage	227
LotArea	0
Street	0
Alley	1352
LotShape	0
LandContour	0
Utilities	2
LotConfig	0
LandSlope	0
Neighborhood	0
Condition1	0
Condition2	0
BldgType	0
HouseStyle	0
OverallQual	0
OverallCond	0
YearBuilt	0
YearRemodAdd	0
RoofStyle	0
RoofMatl	0
Exterior1st	1
Exterior2nd	1
MasVnrType	16
MasVnrArea	15
ExterQual	0
ExterCond	0
Foundation	0
...	
HalfBath	0
BedroomAbvGr	0
KitchenAbvGr	0
KitchenQual	1
TotRmsAbvGrd	0
Functional	2
Fireplaces	0
FireplaceQu	730
GarageType	76
GarageYrBlt	78
GarageFinish	78
GarageCars	1
GarageArea	1
GarageQual	78
GarageCond	78
PavedDrive	0

```

WoodDeckSF      0
OpenPorchSF     0
EnclosedPorch   0
3SsnPorch       0
ScreenPorch     0
PoolArea        0
PoolQC         1456
Fence           1169
MiscFeature     1408
MiscVal         0
MoSold          0
YrSold          0
SaleType        1
SaleCondition   0
Length: 80, dtype: int64

```

```
sns.heatmap(test.isnull())
```



Handling NULL data

We can see that 'Alley', 'PoolQC', 'Fence' and 'MiscFeature' columns have more than 70% of null values in both train and test data. So, we will drop these columns.

Also, we will drop 'Id' column. For non-categorical columns, we will handle null values by filling mean of the column.

For categorical columns, we will handle null values by filling mode of the column.

For Train Data

```
cat_col_train = ['FireplaceQu', 'GarageType', 'GarageFinish', 'MasVnrType', 'BsmtQual',
                 'BsmtCond', 'BsmtExposure', 'BsmtFinType1', 'BsmtFinType2', 'FireplaceQu',
                 'GarageQual', 'GarageCond']
```

```
ncat_col_train = ['LotFrontage', 'GarageYrBlt', 'MasVnrArea']
```

```
for i in cat_col_train:
    train[i] = train[i].fillna(train[i].mode()[0])
```

```
for j in ncat_col_train:
    train[j] = train[j].fillna(train[j].mean())
```

For Test Data

```
cat_col_test = ['FireplaceQu', 'GarageType', 'GarageFinish', 'MasVnrType', 'BsmtQual',
                'BsmtCond', 'BsmtExposure', 'BsmtFinType1', 'BsmtFinType2', 'FireplaceQu',
```

```
'GarageQual', 'GarageCond', 'MSZoning', 'Utilities', 'Exterior1st', 'Exterior2nd', 'KitchenQual', 'Functional', 'SaleType']
```

```
ncat_col_test =
['LotFrontage', 'GarageYrBlt', 'MasVnrArea', 'BsmtFinSF1', 'BsmtFinSF2', 'BsmtUnfSF', 'TotalBsmtSF', 'BsmtFullBath',
 'BsmtHalfBath', 'GarageCars', 'GarageArea']
```

```
for i in cat_col_test:
    test[i] = test[i].fillna(test[i].mode()[0])
```

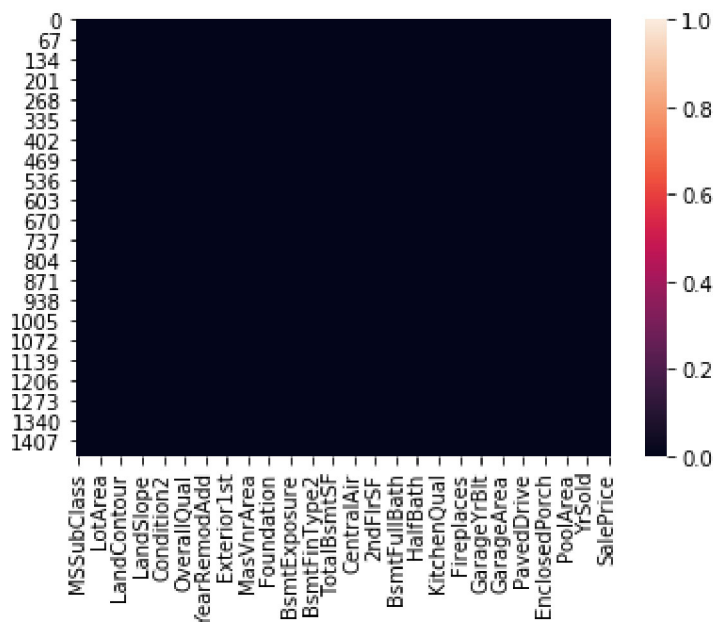
```
for j in ncat_col_test:
    test[j] = test[j].fillna(test[j].mean())
```

Drop Columns

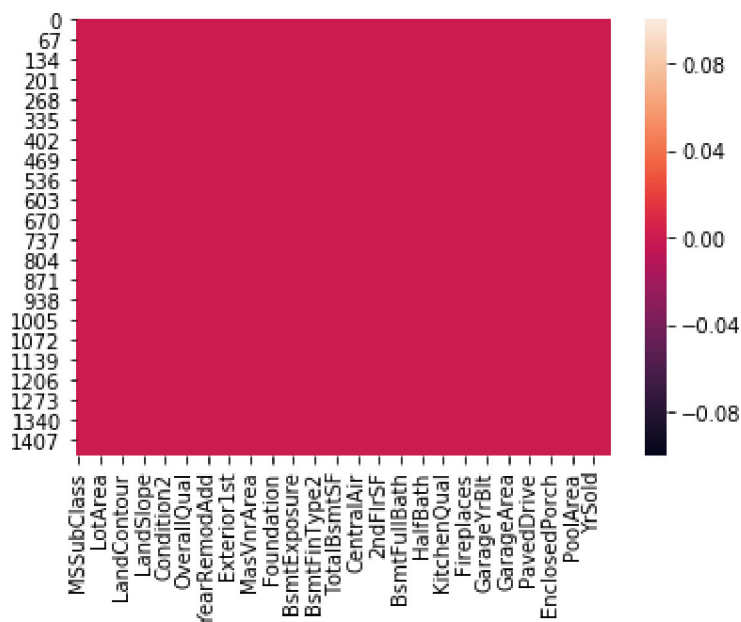
```
to_drop = ['Id', 'Alley', 'PoolQC', 'Fence', 'MiscFeature']
```

```
for k in to_drop:
    train.drop([k], axis = 1, inplace = True)
    test.drop([k], axis = 1, inplace = True)
```

```
sns.heatmap(train.isnull())
```



```
sns.heatmap(test.isnull())
```



```
print("Train Shape: ",train.shape)
print("Test Shape: ",test.shape)
```

Train Shape: (1460, 76)

Test Shape: (1459, 75)

It is observed that for some columns in train data few categories are not present but available in test data. So, we will concat test data to train data, then perform one hot encoding on all categorical columns.

```
final_df = pd.concat([train,test], axis = 0)
final_df.shape
```

(2919, 76)

```
all_cat_col = ['MSZoning', 'Street', 'LotShape', 'LandContour', 'Utilities', 'LotConfig', 'LandSlope',
               'Neighborhood', 'Condition1', 'Condition2', 'BldgType', 'HouseStyle', 'RoofStyle', 'RoofMatl',
               'Exterior1st', 'Exterior2nd', 'MasVnrType', 'ExterQual', 'ExterCond', 'Foundation', 'BsmtQual',
               'BsmtCond', 'BsmtExposure', 'BsmtFinType1', 'BsmtFinType2', 'Heating', 'HeatingQC', 'CentralAir',
               'Electrical', 'KitchenQual', 'Functional', 'FireplaceQu', 'GarageType', 'GarageFinish', 'GarageQual',
               'GarageCond', 'PavedDrive', 'SaleType', 'SaleCondition']
```

```
def cat_onehot_encoding(multicol):
    df_final = final_df
    i = 0
    for fields in multicol:
        print(fields)
        df1 = pd.get_dummies(final_df[fields], drop_first = True)

        final_df.drop([fields], axis = 1, inplace = True)
        if i==0:
            df_final = df1.copy()
        else:
            df_final = pd.concat([df_final, df1], axis=1)
        i = i+1

    df_final = pd.concat([final_df, df_final], axis = 1)

    return df_final
```

```
final_df = cat_onehot_encoding(all_cat_col)
```

MSZoning
Street
LotShape
LandContour
Utilities
LotConfig
LandSlope
Neighborhood
Condition1
Condition2
BldgType
HouseStyle
RoofStyle
RoofMatl
Exterior1st
Exterior2nd
MasVnrType
ExterQual
ExterCond
Foundation
BsmtQual
BsmtCond
BsmtExposure
BsmtFinType1
BsmtFinType2
Heating
HeatingQC
CentralAir
Electrical
KitchenQual
Functional
FireplaceQu
GarageType
GarageFinish
GarageQual
GarageCond
PavedDrive
SaleType
SaleCondition

final_df.shape

(2919, 237)

```
final_df = final_df.loc[:,~final_df.columns.duplicated()]  
final_df.shape
```

(2919, 177)

Let's now divide our data to train and test data.

```
df_train = final_df.iloc[:1460,:]  
df_test = final_df.iloc[1460:,:]  
  
df_test.drop(['SalePrice'], axis = 1, inplace = True)  
  
print("Train Shape: ",df_train.shape)  
print("Test Shape: ",df_test.shape)
```

Train Shape: (1460, 177)

Test Shape: (1459, 176)

Training Data

```
x_train = df_train.drop(['SalePrice'], axis = 1)  
y_train = df_train['SalePrice']
```

XGBoost

```
import xgboost  
  
xgb_model = xgboost.XGBRegressor()  
xgb_model.fit(x_train, y_train)
```

```
XGBRegressor(base_score=0.5, booster='gbtree', colsample_bylevel=1,
             colsample_bynode=1, colsample_bytree=1, gamma=0, gpu_id=-1,
             importance_type='gain', interaction_constraints='',
             learning_rate=0.300000012, max_delta_step=0, max_depth=6,
             min_child_weight=1, missing=nan, monotone_constraints='()',
             n_estimators=100, n_jobs=0, num_parallel_tree=1,
             objective='reg:squarederror', random_state=0, reg_alpha=0,
             reg_lambda=1, scale_pos_weight=1, subsample=1, tree_method='exact',
             validate_parameters=1, verbosity=None)
```

```
param = {
    'n_estimators': [100, 500, 900, 1100, 1500],
    'max_depth': [2, 3, 5, 10, 15],
    'learning_rate': [0.05, 0.1, 0.15, 0.2],
    'min_child_weight': [1, 2, 3, 4],
    'booster': ['gbtree', 'gblinear'],
    'base_score': [0.25, 0.5, 0.75, 1]
}
```

```
from sklearn.model_selection import RandomizedSearchCV
```

```
random_cv = RandomizedSearchCV(estimator=xgb_model,
                               param_distributions = param,
                               cv=5, n_iter=50,
                               scoring = 'neg_mean_absolute_error', n_jobs = 4,
                               verbose = 5,
                               return_train_score = True,
                               random_state = 42)

random_cv.fit(x_train, y_train)
```

Fitting 5 folds for each of 50 candidates, totalling 250 fits

```
[Parallel(n_jobs=4)]: Using backend LokyBackend with 4 concurrent workers.
```

```
[Parallel(n_jobs=4)]: Done 10 tasks      | elapsed: 44.3s
```

```
[Parallel(n_jobs=4)]: Done 64 tasks      | elapsed: 6.9min
```

```
[Parallel(n_jobs=4)]: Done 154 tasks     | elapsed: 13.1min
```

```
[Parallel(n_jobs=4)]: Done 250 out of 250 | elapsed: 18.7min finished
```

```
RandomizedSearchCV(cv=5, error_score='raise-deprecating',
                   estimator=XGBRegressor(base_score=0.5, booster='gbtree', colsample_bylevel=1,
                                           colsample_bynode=1, colsample_bytree=1, gamma=0, gpu_id=-1,
                                           importance_type='gain', interaction_constraints='',
                                           learning_rate=0.300000012, max_delta_step=0, max_depth=6,
                                           min_child_weight=1, miss... scale_pos_weight=1, subsample=1, tree_method='exact',
                                           validate_parameters=1, verbosity=None),
                   fit_params=None, iid='warn', n_iter=50, n_jobs=4,
                   param_distributions={'n_estimators': [100, 500, 900, 1100, 1500], 'max_depth': [2, 3, 5,
10, 15], 'learning_rate': [0.05, 0.1, 0.15, 0.2], 'min_child_weight': [1, 2, 3, 4], 'booster':
['gbtree', 'gblinear'], 'base_score': [0.25, 0.5, 0.75, 1]}},
                   pre_dispatch='2*n_jobs', random_state=42, refit=True,
                   return_train_score=True, scoring='neg_mean_absolute_error',
                   verbose=5)
```

```
random_cv.best_estimator_
```

```
XGBRegressor(base_score=0.25, booster='gbtree', colsample_bylevel=1,
             colsample_bynode=1, colsample_bytree=1, gamma=0, gpu_id=-1,
             importance_type='gain', interaction_constraints='',
             learning_rate=0.1, max_delta_step=0, max_depth=2,
             min_child_weight=1, missing=nan, monotone_constraints='()',
             n_estimators=900, n_jobs=0, num_parallel_tree=1,
             objective='reg:squarederror', random_state=0, reg_alpha=0,
             reg_lambda=1, scale_pos_weight=1, subsample=1, tree_method='exact',
             validate_parameters=1, verbosity=None)
```

```
xgb_model = xgboost.XGBRegressor(base_score=0.25, booster='gbtree', colsample_bylevel=1,
    colsample_bynode=1, colsample_bytree=1, gamma=0, gpu_id=-1,
    importance_type='gain', interaction_constraints='',
    learning_rate=0.1, max_delta_step=0, max_depth=2,
    min_child_weight=1, monotone_constraints='()',
    n_estimators=900, n_jobs=0, num_parallel_tree=1,
    objective='reg:squarederror', random_state=0, reg_alpha=0,
    reg_lambda=1, scale_pos_weight=1, subsample=1, tree_method='exact',
    validate_parameters=1, verbosity=None)

xgb_model.fit(x_train, y_train)
```

```
XGBRegressor(base_score=0.25, booster='gbtree', colsample_bylevel=1,
    colsample_bynode=1, colsample_bytree=1, gamma=0, gpu_id=-1,
    importance_type='gain', interaction_constraints='',
    learning_rate=0.1, max_delta_step=0, max_depth=2,
    min_child_weight=1, missing=nan, monotone_constraints='()',
    n_estimators=900, n_jobs=0, num_parallel_tree=1,
    objective='reg:squarederror', random_state=0, reg_alpha=0,
    reg_lambda=1, scale_pos_weight=1, subsample=1, tree_method='exact',
    validate_parameters=1, verbosity=None)
```

Save Model

```
f = "xgb_model.pkl"
pickle.dump(xgb_model, open(f, 'wb'))
```

Predictions

```
pred_xgb = xgb_model.predict(df_test)
print(pred_xgb.shape)
```

(1459,)

Submission

```
sub_df = pd.read_csv('sample_submission.csv')
sub_df['SalePrice'] = pred_xgb
sub_df.to_csv('sample_sub_xgb.csv', index = False)
```

Decision Tree

```
from sklearn.tree import DecisionTreeClassifier
```

```
dt_model = DecisionTreeClassifier()  
dt_model.fit(x_train, y_train)
```

```
DecisionTreeClassifier(class_weight=None, criterion='gini', max_depth=None,  
                        max_features=None, max_leaf_nodes=None,  
                        min_impurity_decrease=0.0, min_impurity_split=None,  
                        min_samples_leaf=1, min_samples_split=2,  
                        min_weight_fraction_leaf=0.0, presort=False, random_state=None,  
                        splitter='best')
```

Predictions

```
pred_dt = dt_model.predict(df_test)  
print(pred_dt.shape)
```

(1459,)

Submissions

```
sub_df = pd.read_csv('sample_submission.csv')  
sub_df['SalePrice'] = pred_dt  
sub_df.to_csv('sample_sub_dt.csv', index = False)
```

Artificial Neural Network

```
import keras  
from keras.models import Sequential  
from keras.layers import Dense, Activation, Dropout
```

```
from keras import backend as k  
def root_mean_squared_error(y_true, y_pred):  
    return k.sqrt(k.mean(k.square(y_pred - y_true)))
```

Model

```

nn_model = Sequential()

nn_model.add(Dense(50, kernel_initializer = 'he_uniform', activation = 'relu', input_dim = 176))
nn_model.add(Dense(25, kernel_initializer = 'he_uniform', activation = 'relu'))
nn_model.add(Dense(50, kernel_initializer = 'he_uniform', activation = 'relu'))
nn_model.add(Dense(1, kernel_initializer = 'he_uniform'))

nn_model.compile(loss = root_mean_squared_error, optimizer = 'Adamax')

nn_model.fit(x_train.values, y_train.values, validation_split = 0.25, batch_size = 10, epochs =
1000)

Epoch 1/1000
110/110 [=====] - 0s 3ms/step - loss: 168013.1406 - val_loss: 115584.3594
Epoch 2/1000
110/110 [=====] - 0s 2ms/step - loss: 86745.9062 - val_loss: 62824.8828
Epoch 3/1000
110/110 [=====] - 0s 2ms/step - loss: 69233.1562 - val_loss: 61616.0312
Epoch 4/1000
110/110 [=====] - 0s 2ms/step - loss: 67056.6250 - val_loss: 60411.9336
Epoch 5/1000
110/110 [=====] - 0s 2ms/step - loss: 63450.6094 - val_loss: 59374.9688
...
Epoch 996/1000
110/110 [=====] - 0s 2ms/step - loss: 18076.2363 - val_loss: 28988.9512
Epoch 997/1000
110/110 [=====] - 0s 2ms/step - loss: 17696.7090 - val_loss: 30076.5703
Epoch 998/1000
110/110 [=====] - 0s 2ms/step - loss: 18076.5645 - val_loss: 29395.1172
Epoch 999/1000
110/110 [=====] - 0s 2ms/step - loss: 18084.8574 - val_loss: 29192.2930
Epoch 1000/1000
110/110 [=====] - 0s 2ms/step - loss: 17750.3438 - val_loss: 29504.0605

nn_model.save('nn_model.h5')

```

Predictions

```

pred_nn = nn_model.predict(df_test)
print(pred_nn.shape)

```

(1459, 1)

Submission

```
sub_df = pd.read_csv('../input/house-prices-advanced-regression-techniques/sample_submission.csv')
sub_df['SalePrice'] = pred_nn
sub_df.to_csv('sample_sub_nn.csv', index = False)
```